🔥 Wstęp do Big Data

Hadoop, Spark i ekosystem Apache — od garażu Google'a do Twojego laptopa

Python w Zastosowaniach Big Data

Pavlo Melnyk

📋 Co nas dziś czeka?

1. 🤔 Czym właściwie jest Big Data? (i dlaczego 3V to za mało)
2. 📜 Krótka historia — od Google'a do Apache
3. 🐘 Hadoop — słoń, który zmienił świat danych
4. ⚡ Apache Spark — szybszy następca MapReduce
5. 🧩 Ekosystem Apache — kto jest kim w tej rodzinie
6. 🏛️ Kto tego używa? Przykłady z prawdziwego świata
7. 💻 Pierwsze kroki z kodem
8. ⚠️ Ograniczenia, wyzwania i pułapki

> 🎯 ***Na koniec wykładu:*** *Będziesz wiedzieć, dlaczego Big Data to nie tylko buzzword, jak działa Hadoop i Spark, i napiszesz swój pierwszy program w PySpark!*

💡 Zanim zaczniemy...

Ile danych generuje ludzkość **każdego dnia**?

📊 Odpowiedź: ~402,74 miliona terabajtów dziennie! 🤯

- 📱 **500 milionów** tweetów dziennie
- ▶️ **720 000 godzin** wideo uploadowanych na YouTube dziennie
- 📧 **333 miliardy** e-maili dziennie
- 🔍 **8,5 miliarda** wyszukiwań Google dziennie
- 🎵 Spotify analizuje **600GB danych dziennie** by Ci polecać piosenki

> *"Gdybyś chciał zapisać dzienną produkcję danych na płytach DVD, stos sięgałby do Księżyca... i z powrotem 🌙"*

🤔 Czym jest Big Data?

Big Data to dane, które są **zbyt duże, zbyt szybkie lub zbyt złożone**, żeby przetworzyć je tradycyjnymi metodami.

Klasyczna definicja — 3V (Doug Laney, 2001):

V	Angielski	Znaczenie
📦	**Volume**	Ogromna ilość danych (TB → PB → EB)
⚡	**Velocity**	Szybkość napływu danych (real-time, streaming)
🎭	**Variety**	Różnorodność formatów (tekst, obraz, JSON, logi...)

🤔 Ale 3V to dopiero początek...

Współcześnie mówi się o **5V**, a nawet **7V**:

V	Znaczenie	Przykład
✅ **Veracity**	Wiarygodność danych	Fake newsy w danych z Twittera
💰 **Value**	Wartość biznesowa	Z 1PB danych → 1 cenny insight
👁️ **Visualization**	Jak to pokazać?	Dashboard zamiast tabelki CSV
🔄 **Variability**	Zmienność w czasie	Black Friday vs zwykły wtorek

> 🤔 ***Pomyślcie:*** *Który z tych "V" jest Waszym zdaniem najważniejszy?*

📐 Skala danych — poczujmy różnicę

```
1 KB   = Ten slajd
1 MB   = Zdjęcie z telefonu 📱
1 GB   = Film w HD 🎬
1 TB   = 500 godzin filmów HD
1 PB   = 1000 TB = Cały Netflix 🍿
1 EB   = 1000 PB = Cały internet (mniej więcej!)
1 ZB   = 1000 EB = Globalna roczna produkcja danych
```

Aktualna globalna datasfera (2025): ~181 zettabajtów 🌍

> 💡 *Big Data zaczyna się tam, gdzie kończy się* `pandas` *na Twoim laptopie* 😅

📜 Historia Big Data — jak to się zaczęło?

Oś czasu 🕐

Rok	Wydarzenie	Kto?
2003	Google publikuje paper o GFS (Google File System)	Google
2004	Google publikuje paper o MapReduce	Jeffrey Dean, Sanjay Ghemawat
2005	Doug Cutting tworzy Hadoop (inspiracja: GFS + MapReduce)	Yahoo!
2006	Hadoop staje się projektem Apache	Apache Foundation
2008	Hadoop przetwarza 1TB w 209 sekund (rekord!)	Yahoo!
2009	Powstaje Apache Spark (UC Berkeley)	Matei Zaharia
2014	Spark pobija rekord sortowania: 100TB w 23 min	Databricks

👨‍💻 Doug Cutting — ojciec Hadoopa 🐘

- Twórca Apache Lucene (wyszukiwarka) i Apache Nutch (webcrawler)
- Zainspirowany paperami Google (GFS + MapReduce)
- Nazwa "Hadoop" pochodzi od **pluszowego żółtego słonia** jego syna! 🧸
- "*Best name ever for a tech project*"

👨‍💻 Matei Zaharia — twórca Sparka ⚡

- Doktorant na UC Berkeley (AMPLab)
- Frustracja: MapReduce był **za wolny** do iteracyjnych algorytmów ML
- Pomysł: trzymajmy dane **w pamięci RAM** zamiast na dysku!
- Spark powstał jako projekt badawczy (2009), Apache od 2013

Skąd nazwa "Hadoop"? Historia pluszowego słonia

W 2006 roku Doug Cutting pracował nad projektem w Yahoo! i potrzebował nazwy dla nowego frameworka.

> *"Mój syn miał wtedy ok. 2 lat i chodził wszędzie z żółtym pluszowym słoniem. Słoń nazywał się Hadoop — tak dziecko próbowało powiedzieć 'elephant'. Uznałem, że to świetna nazwa — krótka, łatwa do wymówienia, w żaden sposób nie kojarzy się z niczym innym."*
> *— Doug Cutting*

🐘 Dlaczego słoń jest idealnym symbolem Hadoopa?

Cecha słonia	Odpowiednik w Hadoop
Doskonała pamięć	Przechowuje dane niezawodnie (HDFS)
Ogromna siła	Przetwarza petabajty danych
Pracuje w stadzie	Klaster węzłów współpracujących
Powolny, ale wytrwały	Batch processing — nie sprint, ale maraton

⚡ Skąd nazwa "Spark"? — Dosłownie: iskra 🔥

Matei Zaharia wybrał nazwę **Spark** z kilku powodów:

1. **Iskra zapala ogień** — Spark miał "zapalić" erę szybkiego Big Data tam, gdzie Hadoop gasł
2. **Spark of inspiration** — projekt zrodził się z frustracji i błysku pomysłu na UC Berkeley
3. **Szybkość** — iskra jest natychmiastowa, tak jak in-memory computing w porównaniu do zapisu na dysk
4. **Kontrast z Hadoopem** — słoń (ciężki, powolny) vs iskra (lekka, błyskawiczna) 💡

⚡ Spark — Ciekawostka z AMPLab 🔬

Spark powstał w ramach projektu badawczego **AMPLab** (Algorithms, Machines, People Lab) na UC Berkeley w 2009. Matei Zaharia pisał go jako **projekt poboczny podczas doktoratu** — i zmienił świat Big Data! 🚀

🎓 Geneza Sparka — frustracja jako napęd innowacji

Rok 2009, kampus UC Berkeley...

```
Problem Mateiego:
"Pracuję nad algorytmami Machine Learning.
 Muszę iterować przez te same dane 100 razy.
 MapReduce za każdą iteracją zapisuje na dysk.
 Jedno trenowanie modelu = GODZINY czekania. 😤"

Pomysł:
"A gdyby trzymać dane w RAM między iteracjami?
 RAM jest 100x szybszy niż dysk.
 100 iteracji × 100x szybciej = 10 000x potencjalny speedup!"

Wynik (2014):
Spark sortuje 100 TB w 23 minuty.
Hadoop (MapReduce) potrzebował 72 minuty.
→ Spark jest 3x szybszy i używa 10x mniej maszyn! 🏆
```


> 💡 *Jedna dobra idea podczas doktoratu → dziś jeden z **top 5 projektów Apache** z dziesiątkami tysięcy kontrybutorów*

🦁 Cały Apache Zoo — dlaczego takie nazwy?

Historia nazewnictwa projektów Apache jest... osobliwa 😄

Legenda głosi, że Doug Cutting (Hadoop) lubił nazwy bez znaczenia — krótkie, łatwe do zapamiętania, niemające skojarzeń w branży. Inni twórcy poszli w jego ślady:

Projekt	Skąd nazwa
🐘 **Hadoop**	Pluszowy słoń syna Cutting'a
🐝 **Hive**	Ul (dane jak pszczoły wracające do ula)
🐷 **Pig**	"Pigs eat anything" — przetwarza dane w każdym formacie
🐿️ **ZooKeeper**	"Koordynuje zoo projektów Apache" — dosłownie żart!
📨 **Kafka**	Jay Kreps: "*Franz Kafka był pisarzem. To system do pisania danych. Brzmi dobrze.*"
⚡ **Spark**	Iskra zapalająca rewolucję Big Data
🌊 **Flume**	Flume = kanał/rynna — transportuje dane jak woda

Kluczowy pomysł: Distributed Computing

Problem

Masz 1PB danych. Jeden komputer przetwarza ~100MB/s.

- **Czas przetwarzania: ~115 dni** 😱

Rozwiązanie

Rozdziel dane na **1000 komputerów**, każdy przetwarza kawałek.

- **Czas przetwarzania: ~2.7 godziny** 🚀

> 🎯 ***To jest cała filozofia Big Data:*** *zamiast kupować większy komputer (vertical scaling), użyj wielu małych (horizontal scaling)!*

🤔 Think About It

Pytanie: Dlaczego nie można po prostu kupić super-komputera za miliony dolarów zamiast używać klastra zwykłych maszyn?

Odpowiedź:

1. 💰 Koszt — 1000 tanich maszyn < 1 superkomputer
2. 🔧 Awarie — jak padnie 1 z 1000, reszta działa. Jak padnie superkomputer...
3. 📈 Skalowalność — potrzebujesz więcej mocy? Dodaj kolejne maszyny!
4. 🏪 Commodity hardware — używamy zwykłych serwerów "z półki"

🐘 Apache Hadoop

Słoń, który nauczył się liczyć

🐘 Czym jest Hadoop?

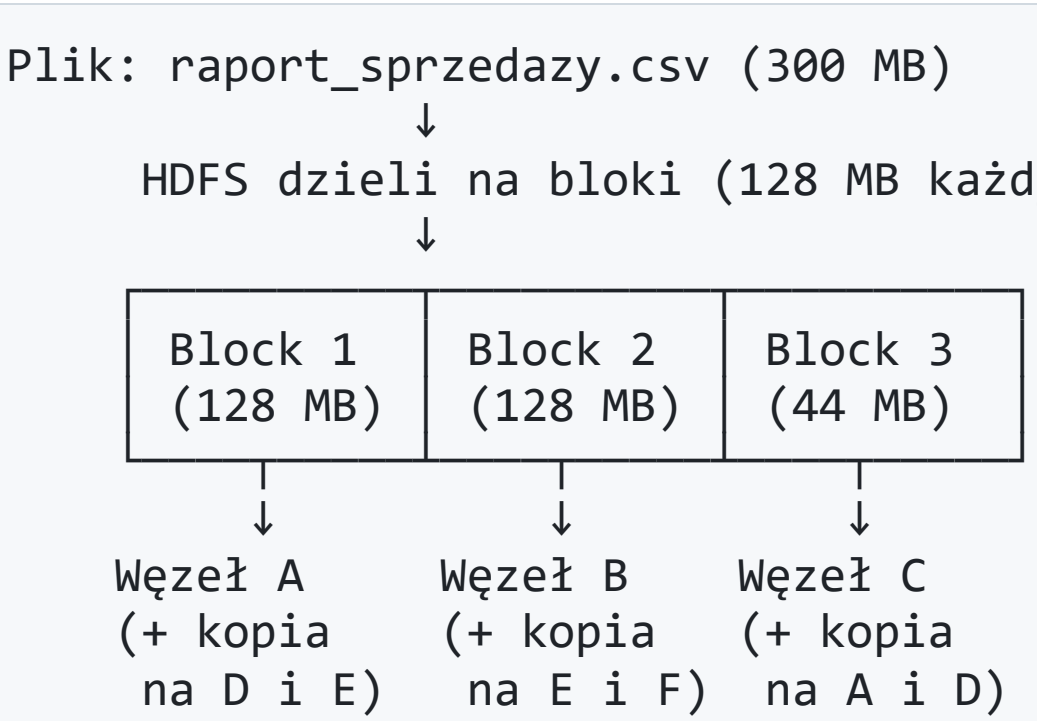
Apache Hadoop — open-source framework do **rozproszonego przechowywania i przetwarzania** ogromnych zbiorów danych na klastrach commodity hardware.

Trzy filary Hadoopa:

Komponent	Rola	Analogia
HDFS	Rozproszony system plików	📁 "Dysk twardy" rozłożony na wiele maszyn
MapReduce	Model przetwarzania	🏭 Linia montażowa do danych
YARN	Zarządzanie zasobami	👔 "Menedżer" klastra

> 💡 *Hadoop = HDFS + MapReduce + YARN*

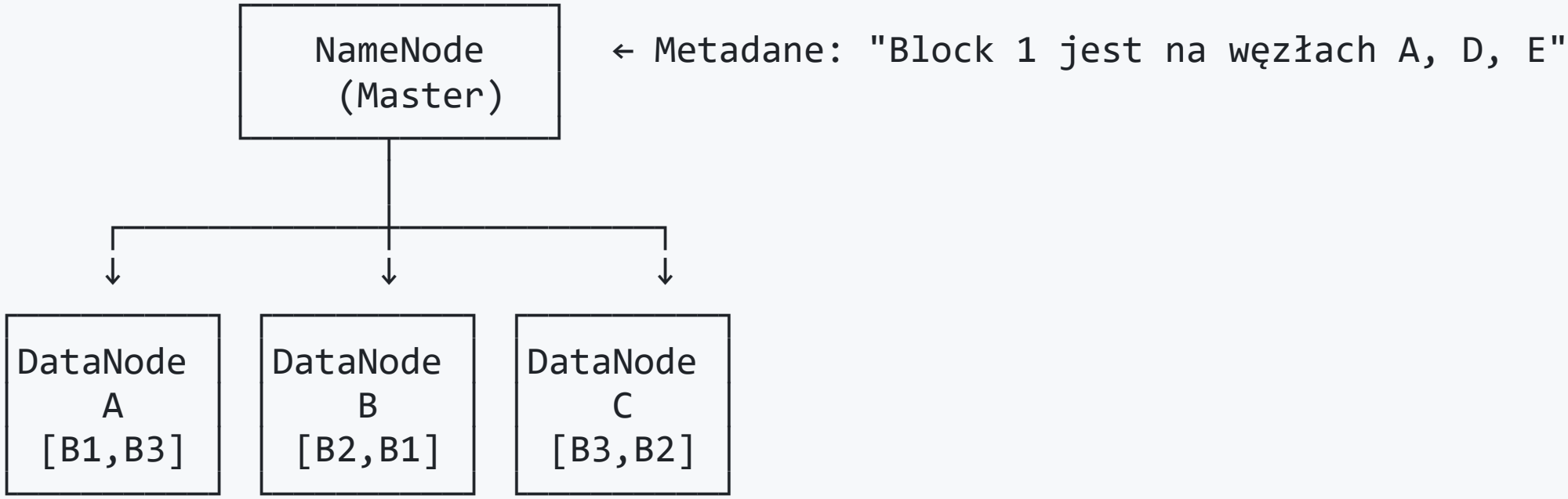
HDFS — podział pliku na bloki



📁 HDFS — replikacja i węzły

- Domyślna **replikacja: 3 kopie** każdego bloku (fault tolerance!)
- **NameNode** — wie, gdzie jest każdy blok (mózg HDFS 🧠)
- **DataNode** — przechowuje bloki danych (mięśnie 💪)

HDFS — architektura



HDFS — kluczowe cechy

- ✅ Fault-tolerant — przetrwa awarię dysków i węzłów
- ✅ Skalowalny — od 3 do **10 000+ węzłów**
- ⚠️ Optymalizowany do **dużych plików** (nie miliardów małych plików!)
- ⚠️ **Write-once, read-many** — nie edytujesz plików "w miejscu"

MapReduce — przetwarzanie danych

Idea: podziel, policz, połącz

```
Tekst: "big data is big and data is everywhere"

MAP (na każdym węźle osobno):
Węzeł A: "big data is big"  → {big:2, data:1, is:1}
Węzeł B: "and data is everywhere" → {and:1, data:1, is:1, everywhere:1}

SHUFFLE (przenieś klucze do odpowiednich węzłów):
Klucz "big"  → Węzeł X
Klucz "data" → Węzeł Y
...

REDUCE (zsumuj wartości dla każdego klucza):
{big:2, data:2, is:2, and:1, everywhere:1}
```

> *Tak Google indeksuje cały internet! Każda strona WWW → Map → Reduce → wynik wyszukiwania*

MapReduce — przykład w Pythonie

Word Count — "Hello World" Big Data

```python
# Faza MAP – przetwarza jedną linię tekstu
def mapper(line: str) -> list[tuple[str, int]]:
    """Dzieli tekst na słowa i emituje (słowo, 1)."""
    words = line.lower().split()
    return [(word, 1) for word in words]  # 👈 emit key-value pairs

# Faza REDUCE – sumuje wartości dla jednego klucza
def reducer(word: str, counts: list[int]) -> tuple[str, int]:
    """Sumuje wystąpienia danego słowa."""
    return (word, sum(counts))  # 👈 agregacja

# Przykład
text = "Big Data is big and Data is everywhere"
mapped = mapper(text)
print(mapped)
# Output: [('big', 1), ('data', 1), ('is', 1), ('big', 1), ...]
```

⚠️ Problem z MapReduce

MapReduce za każdym razem zapisuje na dysk! 💾

```
Iteracja 1:  Dysk → Map → Dysk → Reduce → Dysk
Iteracja 2:  Dysk → Map → Dysk → Reduce → Dysk
Iteracja 3:  Dysk → Map → Dysk → Reduce → Dysk
                    ...
Iteracja N:  Dysk → Map → Dysk → Reduce → Dysk
```

Dla algorytmów ML (np. gradient descent — setki iteracji):

- Każda iteracja = zapis i odczyt z dysku 😩
- **Dysk jest ~100x wolniejszy niż RAM**

> 🤔 *Czy nie lepiej trzymać dane w pamięci?*
> *Tak pomyślał Matei Zaharia... i stworzył Sparka!* ⚡

⚡ Apache Spark

Kiedy Hadoop nie nadąża

⚡ Czym jest Apache Spark?

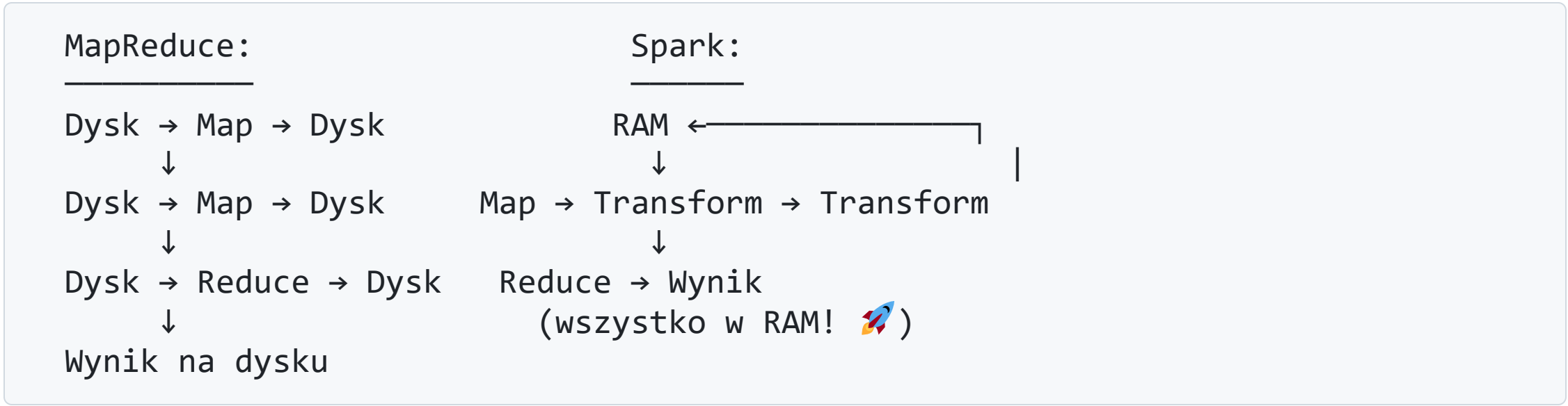
Apache Spark — silnik do rozproszonego przetwarzania danych, działający **w pamięci RAM** (in-memory computing).

Spark vs MapReduce

Cecha	MapReduce 🐢	Spark ⚡
Przechowywanie pośrednie	Dysk	**RAM**
Szybkość	Bazowa	**10-100x szybszy**
Łatwość programowania	Trudne (Java)	**Łatwe (Python/Scala)**
Iteracyjne algorytmy	Tragedia	**Świetne (ML!)**
Streaming	Brak natywnego	**Spark Streaming**
SQL	Hive (wolne)	**Spark SQL**

> 💡 *Spark nie zastępuje HDFS! Spark to **silnik obliczeniowy**, HDFS to **magazyn danych**.*

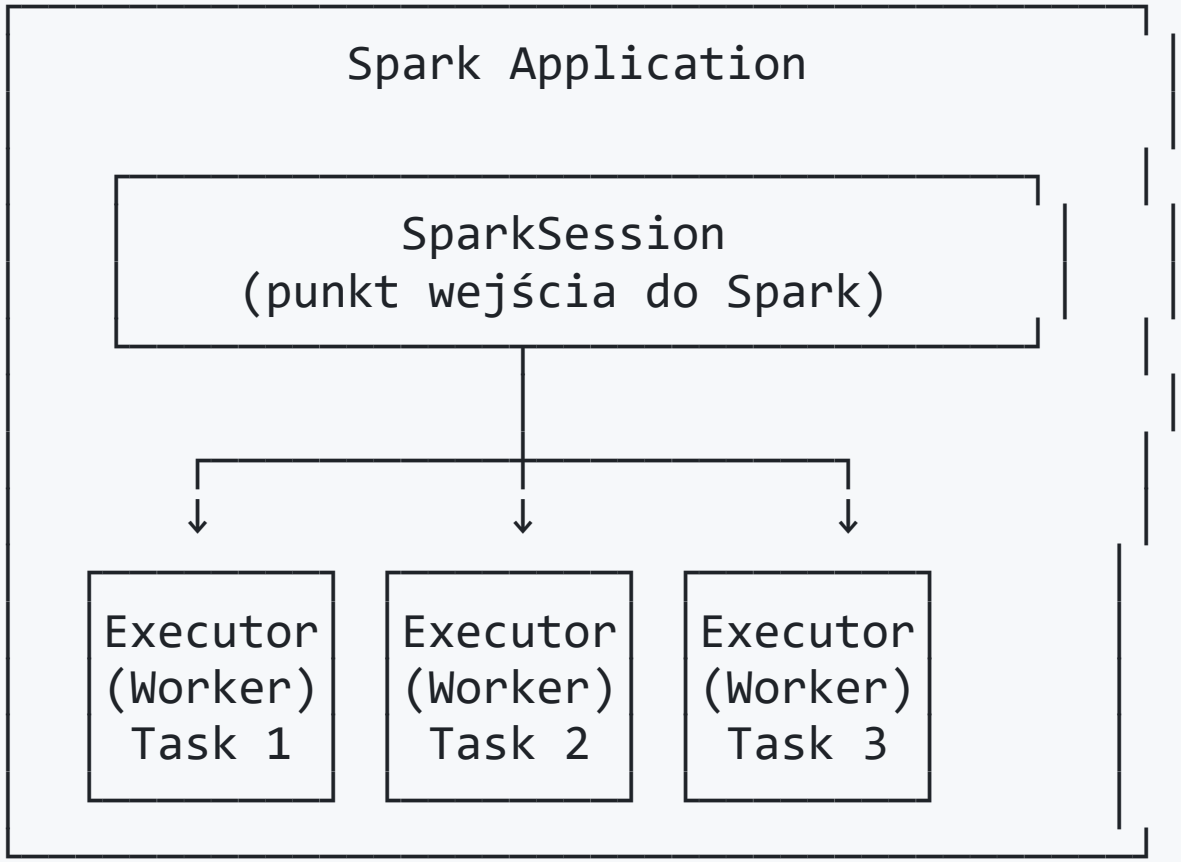
⚡ Jak Spark trzyma dane w pamięci?



Kluczowa innowacja: RDD (Resilient Distributed Dataset)

- Rozproszona kolekcja danych w pamięci
- **Odporna na awarie** (resilient) — Spark pamięta "przepis" na odtworzenie danych
- **Leniwa ewaluacja** (lazy evaluation) — liczy dopiero gdy trzeba

Architektura Spark



Architektura Spark — składniki

- **Driver** — Twój program, koordynuje pracę
- **Executor** — procesy na węzłach klastra, robią "brudną robotę"
- **SparkSession** — punkt wejścia do WSZYSTKIEGO w Spark 2+

💻 PySpark — pierwszy program!

```
from pyspark.sql import SparkSession

# 1 Tworzymy SparkSession – "brama do Sparka"
spark = SparkSession.builder \
    .appName("MojPierwszyProgram") \
    .master("local[*]") \
    .getOrCreate()  # 👈 local[*] = użyj wszystkich rdzeni CPU

# 2 Tworzymy DataFrame (jak pandas, ale rozproszony!)
data = [
    ("Warszawa", 1_800_000, "Mazowieckie"),
    ("Kraków", 800_000, "Małopolskie"),
    ("Łódź", 670_000, "Łódzkie"),
    ("Wrocław", 640_000, "Dolnośląskie"),
    ("Poznań", 530_000, "Wielkopolskie"),
]
columns = ["miasto", "populacja", "wojewodztwo"]
df = spark.createDataFrame(data, columns)

df.show()
```

💻 PySpark — wynik

```
+--------+---------+-------------+
|  miasto|populacja| wojewodztwo |
+--------+---------+-------------+
|Warszawa|  1800000| Mazowieckie |
|  Kraków|   800000| Małopolskie |
|    Łódź|   670000|     Łódzkie |
| Wrocław|   640000| Dolnośląskie|
|  Poznań|   530000|Wielkopolskie|
+--------+---------+-------------+
```

Wygląda znajomo? Tak — **Spark DataFrame ≈ pandas DataFrame**, ale:

- 🌐 Rozproszony na wiele maszyn
- 📈 Obsługuje terabajty danych
- ⚡ Przetwarzanie równoległe
- 🦥 Lazy evaluation — optymalizuje plan przed wykonaniem

💻 PySpark — operacje na DataFrame

```python
from pyspark.sql import functions as F

# Filtrowanie — miasta > 700k mieszkańców
duze_miasta = df.filter(F.col("populacja") > 700_000)
duze_miasta.show()
# +--------+---------+------------+
# |  miasto|populacja|wojewodztwo |
# +--------+---------+------------+
# |Warszawa|  1800000|Mazowieckie |
# |  Kraków|   800000|Małopolskie |
# +--------+---------+------------+

# Agregacja — średnia populacja
avg_pop = df.agg(F.avg("populacja").alias("srednia_populacja"))
avg_pop.show()
# +------------------+
# |srednia_populacja  |
# +------------------+
# |          888000.0 |
# +------------------+
```

💻 PySpark — Spark SQL

```
# Rejestrujemy DataFrame jako tabelę SQL
df.createOrReplaceTempView("miasta")

# Teraz możemy pisać SQL! 🎉
wynik = spark.sql("""
    SELECT wojewodztwo,
           COUNT(*) as liczba_miast,
           SUM(populacja) as laczna_populacja
    FROM miasta
    GROUP BY wojewodztwo
    ORDER BY laczna_populacja DESC
""")

wynik.show()
# +-------------+------------+----------------+
# | wojewodztwo |liczba_miast|laczna_populacja|
# +-------------+------------+----------------+
# | Mazowieckie |           1|         1800000|
# | Małopolskie |           1|          800000|
# +-------------+------------+----------------+
```

🐼 pandas vs PySpark — porównanie

```python
# pandas 🐼
import pandas as pd

df = pd.read_csv("dane.csv")


df.head()

df["nowa"] = df["a"] + df["b"]


df.groupby("kat").mean()

df.to_csv("wynik.csv")

len(df)
```

```python
# PySpark ⚡
from pyspark.sql import functions as F

df = spark.read.csv("dane.csv",
     header=True, inferSchema=True)

df.show(5)

df = df.withColumn("nowa",
     F.col("a") + F.col("b"))

df.groupBy("kat").avg()

df.write.csv("wynik.csv")

df.count()
```

> 🤝 *API jest bardzo podobne! Jeśli umiesz pandas — PySpark będzie naturalny.*

🤔 Kiedy pandas, a kiedy Spark?

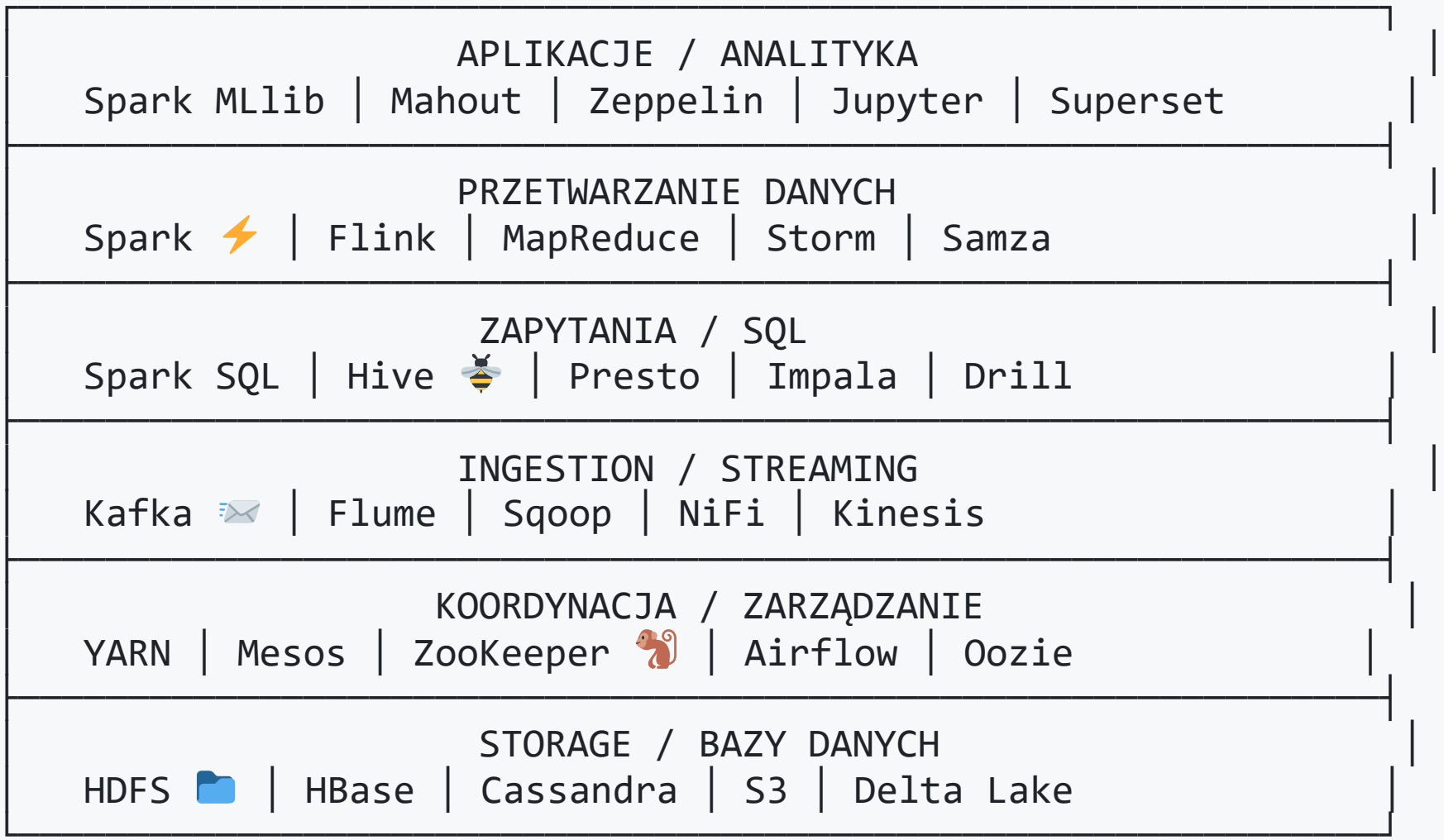
Kryterium	pandas 🐼	PySpark ⚡
Rozmiar danych	< 10 GB	> 10 GB
Komputer	Twój laptop	Klaster / chmura
Szybkość prototypowania	Szybsza	Wolniejsza (setup)
Operacje ML iteracyjne	scikit-learn	Spark MLlib
Real-time streaming	❌	✅ Spark Streaming
Koszt	Darmowy (1 maszyna)	Klaster = 💰

> 🎯 ***Golden Rule:*** *Zacznij od pandas. Kiedy boli — przejdź na Sparka.*

🧩 Ekosystem Apache

Rodzina jest duża 👨‍👩‍👧‍👦

🧩 Ekosystem Apache — Big Picture



Apache Hive — SQL dla Big Data

Hive = pisz SQL, a Hive przetłumaczy to na MapReduce/Spark

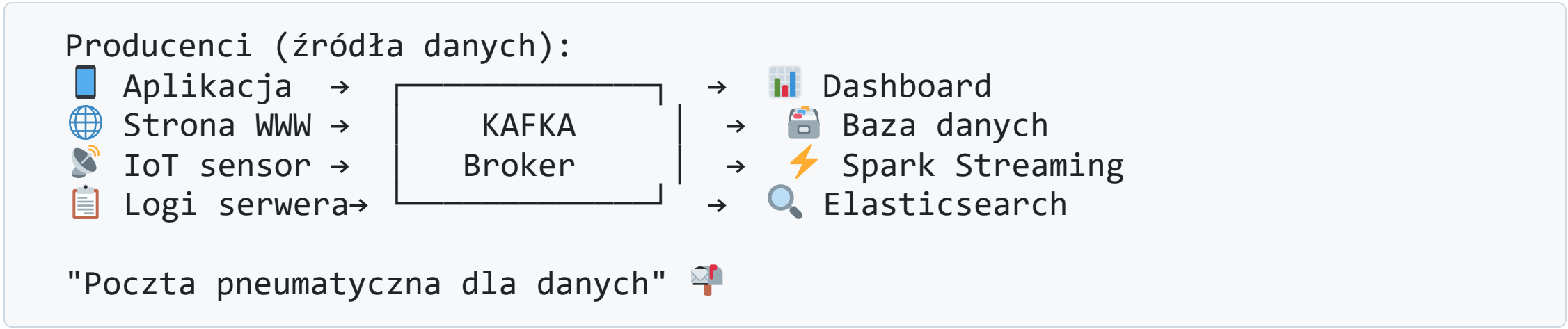
```sql
-- Wygląda jak zwykły SQL!
CREATE TABLE sprzedaz (
    produkt STRING,
    kwota DOUBLE,
    data STRING
) STORED AS PARQUET;

SELECT produkt, SUM(kwota) as laczna_sprzedaz
FROM sprzedaz
WHERE data >= '2025-01-01'
GROUP BY produkt
ORDER BY laczna_sprzedaz DESC
LIMIT 10;
```

> 💡 *Hive sprawił, że analitycy SQL mogli korzystać z Hadoopa **bez pisania Javy**!*

📨 Apache Kafka — streaming w real-time

Kafka = rozproszona platforma do przesyłania danych w czasie rzeczywistym (message broker)



Fun fact: Kafka przetwarza **biliony (trillions!)** wiadomości dziennie w LinkedIn i Netflix!

Nazwa: Jay Kreps (twórca) nazwał projekt od Franza Kafki, bo... "dobrze brzmi dla systemu do pisania" 📚

🐿️ Apache ZooKeeper — koordynator klastra

ZooKeeper — serwis do koordynacji rozproszonych systemów

```
"Kim jest ZooKeeper? Strażnikiem ogrodu zoologicznego!" 🐘🐝🐿️

Co robi:
├── 🔒 Distributed locking – kto aktualnie "trzyma" zasoby
├── 🗳️ Leader election – który węzeł jest szefem
├── 📋 Configuration management – wspólna konfiguracja
└── 🔔 Notification – "hej, coś się zmieniło!"
```

Używany przez: Kafka, HBase, Hadoop, Solr...

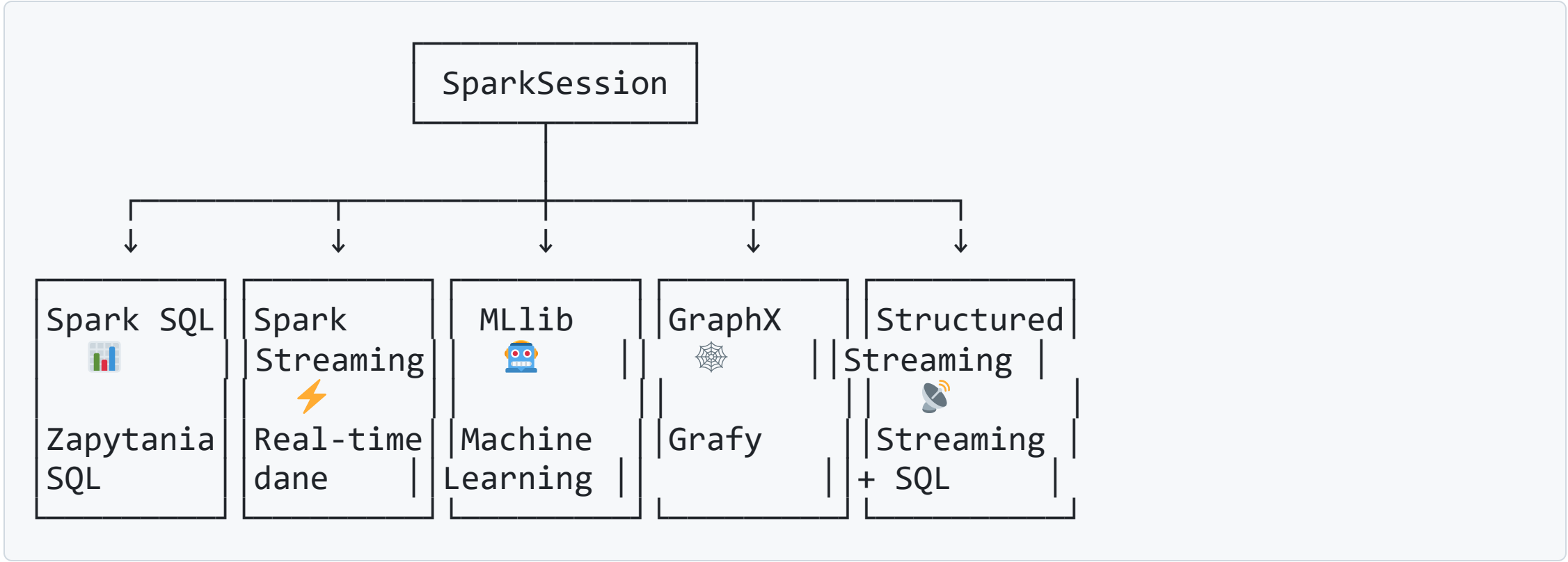
> 🤭 ***Ciekawostka:*** *Nazwy w ekosystemie Apache to prawdziwe ZOO — Hadoop (słoń), Hive (pszczoła), Pig (świnia), ZooKeeper, Impala (antylopa)...*

🦁 Ekosystem zoologiczny Apache

Projekt	Zwierzę / Nazwa	Co robi?
🐘 **Hadoop**	Słoń	Rozproszone przetwarzanie + storage
🐝 **Hive**	Ul / Pszczoła	SQL na Big Data
🐷 **Pig**	Świnia	Skryptowy język do przetwarzania
🐿️ **ZooKeeper**	Opiekun zoo	Koordynacja klastra
🦅 **Impala**	Antylopa	Szybkie zapytania SQL
🦑 **Sqoop**	SQL-to-Hadoop	Import/export danych z RDBMS
🌊 **Flume**	Rzeka / Strumień	Zbieranie logów
⚡ **Spark**	Iskra	In-memory computing
📨 **Kafka**	Franz Kafka 📚	Message streaming

> 😄 *Rozmowa kwalifikacyjna: "Znam Sparka, Kafkę i Świnię" brzmi dziwnie, ale oznacza dużą pensję!*

🧩 Spark Ecosystem — niesamowita wszechstronność



Jeden framework — **pięć bibliotek** — spójne API! 🎯

📊 Formaty danych w Big Data

Format	Typ	Czytelny?	Kompresja	Użycie
CSV	Wierszowy	✅ Tak	❌ Słaba	Import/export
JSON	Wierszowy	✅ Tak	❌ Słaba	API, logi
Avro	Wierszowy	❌ Binarny	✅ Dobra	Kafka, streaming
Parquet	**Kolumnowy**	❌ Binarny	✅✅ Świetna	**Analityka!** 🏆
ORC	**Kolumnowy**	❌ Binarny	✅✅ Świetna	Hive
Delta	Kolumnowy+	❌ Binarny	✅✅ Świetna	Lakehouse

> 🎯 ***Pro tip:*** *W 90% przypadków analitycznych Big Data →* ***Parquet*** *to Twój najlepszy przyjaciel!*

📊 Dlaczego Parquet jest taki świetny?

CSV (wierszowy):

imię	wiek	miasto
Jan	25	WAW
Ola	30	KRK
...	...	...

Parquet (kolumnowy):

```
┌──────────────────────┐
│imię: [Jan, Ola..]    │
│wiek: [25, 30..]      │
│miasto:[WAW,KRK..]    │
└──────────────────────┘
```

```
SELECT AVG(wiek) FROM tabela;
```

CSV: Musi przeczytać WSZYSTKO ❌ (imię, wiek, miasto...)

Parquet: Czyta TYLKO kolumnę "wiek" ✅ (pomija resztę → szybciej!)

> 💡 *Przy 100 kolumnach i zapytaniu o 3 z nich → Parquet czyta **30x mniej danych**!*

🏙️ Kto tego używa?

Przykłady z prawdziwego świata

Big Data w akcji — wielkie firmy

Firma	Technologia	Co robią?
Netflix	Spark, Kafka, Presto	Rekomendacje filmów, A/B testy, ~700 Spark jobs/dzień
Spotify	Spark, Kafka, GCS	Discover Weekly (ML), 600GB danych dziennie
Allegro	Spark, Kafka, Hadoop	Rekomendacje, analityka, wykrywanie fraudów
Uber	Spark, Kafka, Hive	Dynamiczne ceny, szacowanie czasu dojazdu
PayPal	Spark, Kafka	Wykrywanie fraudów w **real-time**
Twitter/X	Hadoop, Storm, Kafka	Przetwarzanie 500M tweetów/dzień
Google	Własne (BigQuery, Dremel)	...dosłownie WSZYSTKO 😄

Netflix — case study

Problem:

Jak polecić filmy 230+ milionom użytkowników w 190 krajach?

Rozwiązanie:

```
Dane o użytkownikach (co oglądali, kiedy, jak długo)
                    ↓
        Apache Kafka (ingestion)
                    ↓
        Apache Spark (przetwarzanie ML)
                    ↓
        Model rekomendacji (collaborative filtering)
                    ↓
        "Polecane dla Ciebie"
```

🍿 Netflix — wyniki w liczbach 📊

- 📊 **1 petabajt** nowych danych dziennie
- ⚡ **700+ Spark jobs** uruchamianych dziennie
- 💰 **80% obejrzanych treści** pochodzi z rekomendacji!
- 💵 System rekomendacji oszczędza Netflix **~1 miliard USD rocznie**

🎵 Spotify — Discover Weekly

Jak Spotify wie, co chcesz słuchać?

```
# Uproszczony schemat (Spark MLlib + Feature Engineering)
from pyspark.ml.recommendation import ALS

# 1. Dane: kto słuchał co i ile razy
ratings = spark.read.parquet("user_song_plays.parquet")

# 2. Model: Alternating Least Squares (collaborative filtering)
als = ALS(
    maxIter=10,                # 👈 iteracyjny algorytm (dlatego Spark!)
    regParam=0.01,
    userCol="user_id",
    itemCol="song_id",
    ratingCol="play_count"
)
model = als.fit(ratings)

# 3. Rekomendacje: top 30 piosenek dla każdego użytkownika
```

🤔 Think About It

Pytanie: Spotify musi przetworzyć dane 600+ milionów użytkowników i milionów piosenek, żeby wygenerować Discover Weekly do poniedziałku rano. Dlaczego Spark (w pamięci) jest tu lepszy niż MapReduce (z dysku)?

Big Data w chmurze

Bo nikt nie chce zarządzać klastrem w piwnicy

☁️ Big Data + Cloud = ❤️

Dlaczego chmura zamiast własnego klastra?

Aspekt	Własny klaster 🏛️	Chmura ☁️
Koszt początkowy	💰💰💰 (serwery)	$0 (pay-as-you-go)
Czas startu	Tygodnie/miesiące	Minuty!
Skalowanie	Kupuj nowe serwery	Kliknij przycisk
Utrzymanie	Zespół adminów	Zarządza dostawca
Elastyczność	Płacisz 24/7	Płacisz za użycie

Popularne usługi Big Data w chmurze

Serwis	Odpowiednik
Google BigQuery	Serverless SQL na petabajtach
AWS EMR	Managed Hadoop/Spark cluster
Azure HDInsight	Managed Hadoop/Spark
Google Dataproc	Managed Spark/Hadoop
Databricks	Unified analytics (Spark++)

💻 BigQuery — Spark w chmurze (bez klastra!)

```
# Google BigQuery – petabajty danych, zero infrastruktury
from google.cloud import bigquery

client = bigquery.Client(project="moj-projekt")

# Zapytanie do publicznego datasetu (Wikipedia pageviews!)
query = """
    SELECT title, SUM(views) as total_views
    FROM `bigquery-public-data.wikipedia.pageviews_2025`
    WHERE DATE(datehour) = '2025-01-01'
      AND wiki = 'pl'
    GROUP BY title
    ORDER BY total_views DESC
    LIMIT 10
"""

result = client.query(query).to_dataframe()  # 👈 zwraca pandas DataFrame!
print(result)
# Najpopularniejsze polskie artykuły Wikipedii 1 stycznia 2025 🅿
```

⚠️ Ograniczenia i wyzwania

Nie wszystko jest złotem co się Big Data świeci

⚠️ Ograniczenia Big Data (1/3)

1. 💰 Koszty

- Klastry to **drogie zabawki** (sprzęt, prąd, ludzie)
- Cloud: łatwo "spalić" budżet zostawiając klaster włączony na noc 😱
- *Przykład:* klaster EMR 10 maszyn × $2/h = **$480/dzień** = **~14 000 PLN/miesiąc**

2. 👨‍💻 Complexity

- Konfiguracja klastra to **nie jest prosta sprawa**
- Debugowanie rozproszonych systemów = **ból głowy** 🤕
- "Działa na moim laptopie" nie znaczy "działa na 100 maszynach"

⚠️ Ograniczenia Big Data (2/3)

3. 📏 Overhead

- Spark ma **narzut startowy** — nie opłaca się do małych danych!
- Setup Sparka do 100MB danych? To jak wynajmowanie TIR-a do przewiezienia pizzy 🍕🚚

4. 🗒️ Small Files Problem

- HDFS nie lubi **milionów małych plików** (NameNode memory!)
- 1 plik = 1 wpis w NameNode → milion plików = problem
- Rozwiązanie: **kompaktowanie** do większych plików (Parquet, Avro)

⚠️ Ograniczenia Big Data (3/3)

5. 🔄 Data Quality

- "Garbage In, Garbage Out" — Big Data **nie naprawi złych danych**
- Brak schema enforcement → chaos w formatach danych
- Rozwiązanie: Delta Lake, Apache Iceberg (schema evolution)

6. 🔒 Prywatność i bezpieczeństwo

- RODO/GDPR — jak usunąć dane jednej osoby z 100PB? 😬
- **"Right to be forgotten"** w systemie, który zaprojektowano jako append-only
- Szyfrowanie na poziomie kolumn (Apache Ranger, Kerberos)

🛠️ Porównanie technologii — kiedy co wybrać?

Scenariusz	Technologia	Dlaczego?
Analityka na laptopie (<10 GB)	**pandas** 🐼	Prosta, szybka, lokalna
Batch processing (TB+)	**Spark** ⚡	In-memory, szybki
Real-time streaming	**Kafka + Spark Streaming**	Event-driven
SQL na petabajtach (chmura)	**BigQuery** ☁️	Serverless, ultra-szybki
ETL / Data pipelines	**Spark + Airflow**	Orchestracja + przetwarzanie
Machine Learning (Big Data)	**Spark MLlib**	Rozproszone trenowanie
Machine Learning (normalne)	**scikit-learn** 🐍	Prostsze, na 1 maszynie

📈 Trendy 2025/2026 — co teraz "hot" w Big Data? 🔥

Trend	Opis
🏠 **Data Lakehouse**	Połączenie Data Lake + Data Warehouse (Delta Lake, Iceberg)
🤖 **AI/ML Pipelines**	MLOps, Spark + ML w produkcji
📡 **Real-time everything**	Od batch do streaming-first
🧊 **Apache Iceberg**	Nowy standard formatu tabel (Netflix open-source!)
☁️ **Serverless Spark**	Databricks Serverless, Dataproc Serverless
🔗 **Data Mesh**	Decentralizacja — domeny odpowiadają za swoje dane
📊 **dbt + Spark**	Analytics Engineering na Big Data

> 🎯 *Umiejętność Big Data = **pewna praca** na najbliższe 10+ lat!*

🤔 Mini-quiz! Sprawdźmy co zapamiętaliście

Q1: Jak nazywa się rozproszony system plików Hadoopa?

Q2: Dlaczego Spark jest szybszy od MapReduce?

Q3: Co oznacza 3V w kontekście Big Data?

Q4: Jak nazywa się kolumnowy format plików, najczęściej używany w analityce?

Q5: Kto stworzył Hadoopa i skąd wzięła się nazwa?

💻 Hands-on: Twój pierwszy Spark job

```
from pyspark.sql import SparkSession, functions as F

# Uruchom na Colab: !pip install pyspark
spark = SparkSession.builder.appName("FirstJob").master("local[*]").getOrCreate()

# Wczytaj dane (CSV z internetu)
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"

# W PySpark wczytujemy tak:
df = spark.read.csv(url, header=True, inferSchema=True)

# 🔍 Analiza: przeżywalność wg klasy i płci
survival = (df.groupBy("Pclass", "Sex")
    .agg(
        F.count("*").alias("liczba_pasazerow"),
        F.avg("Survived").alias("procent_przezycia"),  # 👈 mean survival rate
        F.avg("Age").alias("sredni_wiek")
    )
    .orderBy("Pclass", "Sex"))

survival.show()
spark.stop()  # 👈 Zawsze zatrzymuj SparkSession!
```

💻 Hands-on: Wynik

```
+------+------+----------------+-----------------+-----------+
|Pclass|   Sex|liczba_pasazerow|procent_przezycia|sredni_wiek|
+------+------+----------------+-----------------+-----------+
|     1|female|              94|             0.97|       34.6|
|     1|  male|             122|             0.37|       41.3|
|     2|female|              76|             0.92|       28.7|
|     2|  male|             108|             0.16|       30.7|
|     3|female|             144|             0.50|       21.8|
|     3|  male|             347|             0.14|       26.5|
+------+------+----------------+-----------------+-----------+
```

Co widzimy? 🚢

- Kobiety z 1 klasy: **97% przeżywalność**
- Mężczyźni z 3 klasy: **14% przeżywalność**
- "Women and children first" — dane to potwierdzają!

📝 Podsumowanie — co dzisiaj odkryliśmy

1. ✅ **Big Data** = dane zbyt duże/szybkie/złożone dla tradycyjnych narzędzi (5V)
2. ✅ **Hadoop** = HDFS (storage) + MapReduce (compute) + YARN (management) 🐘
3. ✅ **Spark** = in-memory computing, 10-100x szybszy od MapReduce ⚡
4. ✅ **Ekosystem Apache** = Kafka, Hive, ZooKeeper + wielu innych 🧩
5. ✅ **Parquet** = kolumnowy format, idealny do analityki 📊
6. ✅ **Cloud** = BigQuery, Dataproc, EMR — Big Data bez własnego klastra ☁️
7. ✅ **Real-world** = Netflix, Spotify, Allegro — wszyscy tego używają 🌍

> 🎯 ***Key takeaway:*** *Big Data to nie magia — to sprytne rozłożenie pracy na wiele maszyn!*

🔮 Co dalej?

Następny wykład: Głębokie zanurzenie w Apache Spark — RDD, DataFrame, Spark SQL 🔥

Do zapoznania się:

- 📖 Spark: The Definitive Guide (Zaharia et al.) — rozdziały 1-3
- 📖 Hadoop: The Definitive Guide (Tom White) — rozdział 1
- 📽️ Matei Zaharia: Origin of Spark (YouTube)

🎁 Bonus: Ciekawostki tematyczne 🎉

- 🃏 **Hadoop** przetworzył dane z **Large Hadron Collider** w CERN
- 🎬 **Netflix** przeniosło WSZYSTKIE dane (60 PB) do chmury AWS w 7 lat
- 📊 **Facebook** ma największy klaster Hadoop na świecie (~300 PB)
- 🤖 **ChatGPT** trenowano m.in. dzięki infrastrukturze Big Data
- 🐘 Istnieje **Hadoop w wersji Raspberry Pi** — klaster z 10 Raspberry Pi 🫐
- 💰 Średnia pensja Big Data Engineer w Polsce (2025): **18 000 - 28 000 PLN/mies.**
- 🌍 Do 2028 globalna datasfera ma osiągnąć **394 zettabajty**

❓ Pytania?

Dziękuję za uwagę! 🎉

> *"Without Big Data, you are blind and deaf and in the middle of a freeway."*
> *— Geoffrey Moore*